

Atlantic City CRM

Manual interno para el equipo de trabajo

Guía completa para clonar, instalar, configurar, ejecutar, administrar y trabajar el proyecto desde GitHub privado

Versión 1.0 - Julio 2026

Importante: Este manual está pensado para el equipo del curso. No es un manual para usuarios finales. La finalidad es que cualquier integrante pueda levantar el proyecto en su equipo sin depender de una sola computadora ni volver al chat para revisar pasos.

Dato	Detalle
Proyecto	Atlantic City CRM
Tipo	MVP funcional académico
Backend	Python + Django
Base de datos	MySQL
Frontend	HTML, CSS, Bootstrap y JavaScript básico
Repositorio	GitHub privado
Responsable de repositorio	anthony-devtecnio
Uso del documento	Guía interna del equipo de trabajo

Índice de trabajo

- 1. Presentación y objetivo del manual
- 2. Qué contiene el sistema Atlantic City CRM
- 3. Cómo se compartirá el proyecto por GitHub privado
- 4. Requisitos previos en Windows 11
- 5. Instalación de herramientas necesarias
- 6. Clonar el repositorio desde GitHub
- 7. Crear y activar el entorno virtual
- 8. Instalar dependencias del proyecto
- 9. Configurar MySQL y la base de datos
- 10. Configurar settings.py
- 11. Ejecutar check, migraciones y servidor
- 12. Crear superusuario e ingresar al admin
- 13. Crear grupos, roles y permisos
- 14. Crear datos de prueba y promociones
- 15. Uso básico del sistema
- 16. Trabajo con Git: commits, push, pull y buenas prácticas
- 17. Errores comunes y soluciones
- 18. Nota para quien trabaja en Mac
- 19. Checklist final para levantar el proyecto
- 20. Cierre y recomendaciones finales
- Referencias de apoyo

1. Presentación y objetivo del manual

Este manual lo preparé para que todos podamos trabajar el proyecto Atlantic City CRM de forma ordenada. La idea es que cada integrante pueda descargar el repositorio privado, instalar lo necesario en su computadora, configurar la base de datos, levantar el sistema y probar los módulos sin depender de mi equipo principal.

El documento está redactado paso a paso, sin asumir que todos ya saben usar Django, MySQL o GitHub. Por eso incluyo comandos, rutas, recomendaciones y errores comunes que pueden aparecer durante la instalación.

***Nota:** No debemos saltarnos pasos. Si un comando muestra error, lo correcto es detenerse, revisar y recién continuar. Avanzar a ciegas puede generar más errores.*

Objetivo	Detalle
Levantar el proyecto	Que cualquier integrante pueda ejecutar el sistema en Windows 11.
Entender el flujo	Saber qué hace cada módulo y cómo se usa.
Trabajar en equipo	Usar GitHub privado para compartir avances y evitar versiones desordenadas.
Administrar datos	Usar Django Admin para crear usuarios, grupos, permisos y promociones.

Evitar errores	Dejar claras las soluciones a problemas frecuentes.
----------------	---

2. Qué contiene el sistema Atlantic City CRM

Atlantic City CRM es un MVP funcional desarrollado para centralizar la información de clientes. El sistema se organizó alrededor de una ficha única del cliente, donde se puede consultar información básica y el historial de visitas, incidencias y promociones.

Módulo	Funcionalidad principal
Login y logout	Permite iniciar y cerrar sesión.
Dashboard	Muestra indicadores generales, estados y últimos registros.
Clientes	Permite registrar, editar, buscar y consultar clientes.
Ficha única	Muestra datos del cliente, visitas, incidencias y promociones asignadas.
Visitas	Registra visitas desde la ficha del cliente y permite ver el historial general.
Incidencias	Registra reclamos, consultas, quejas o problemas asociados al cliente.
Promociones	Permite crear promociones y asignarlas a clientes.
Roles y permisos	Controla qué puede ver o hacer cada tipo de usuario.
Página 403	Muestra acceso no permitido cuando un usuario no tiene permisos.

3. Forma oficial de compartir el proyecto: GitHub privado

El proyecto se compartirá mediante un repositorio privado en GitHub. Esta será la forma oficial de trabajo para el equipo, porque nos permite mantener el código protegido y dar acceso solamente a los integrantes autorizados.

Decisión	Motivo
Repositorio privado	Evita que personas externas ingresen al código.
Acceso solo a 4 integrantes	Permite trabajar en equipo sin exponer el proyecto.
GitHub como fuente principal	Todos trabajan sobre la misma versión.
Drive solo como respaldo	Puede usarse para compartir manuales o archivos finales, no como control principal del código.

Importante: Aunque el repositorio sea privado, no debemos subir contraseñas personales, archivos .env, la carpeta venv ni bases de datos locales.

3.1 Repositorio del proyecto

El repositorio se creó con el usuario de GitHub:

```
anthony-devtecnio
```

Nombre del repositorio:

```
atlantic_city_crm
```

La URL del repositorio es:

```
https://github.com/anthony-devtecnio/atlantic_city_crm.git
```

3.2 Qué sí debe subirse al repositorio

- Código fuente del proyecto.
- Carpetas de las apps: clientes, visitas, incidencias, promociones, dashboard.
- Carpeta config.
- Plantillas dentro de templates.
- Archivos estáticos dentro de static.
- manage.py.
- requirements.txt.
- .gitignore.
- README.md y manuales del equipo, si se decide incluirlos.

3.3 Qué no debe subirse al repositorio

- La carpeta venv.
- Archivos __pycache__.
- Archivos .pyc.
- Archivos .env con contraseñas.
- Base de datos local db.sqlite3.
- Archivos temporales.
- Contraseñas reales o personales.
- Capturas con datos sensibles.

```
# .gitignore recomendado
venv/
__pycache__/
*.pyc
*.pyo
*.pyd
.env
.DS_Store
Thumbs.db
db.sqlite3
*.log
.vscode/
```

3.4 Invitar integrantes al repositorio privado

Después de crear el repositorio privado, se debe invitar únicamente a los integrantes del equipo. La ruta general en GitHub es:

Repositorio > Settings > Collaborators > Add people

Cada integrante debe enviar su usuario de GitHub. Luego se agrega como colaborador y debe aceptar la invitación. Mientras no acepte, no podrá clonar el repositorio privado.

***Nota:** Para este proyecto, el acceso recomendado para los integrantes es Write, porque permite clonar, descargar cambios y subir avances. No se recomienda dar Admin salvo que sea necesario.*

3.5 Preparar y subir el proyecto por primera vez

Estos comandos son para la persona que sube el proyecto por primera vez desde su computadora. Se ejecutan dentro de la carpeta `atlantic_city_crm`.

```
git init
git status
git add .
git commit -m "Primera version MVP Atlantic City CRM"
git branch -M main
git remote add origin https://github.com/anthony-devtecnio/atlantic_city_crm.git
git push -u origin main
```

Importante: Antes de usar `git add .`, siempre revisar `git status` para confirmar que no aparezca `venv`, `.env`, `db.sqlite3` o archivos temporales.

4. Requisitos previos en Windows 11

Cada integrante del equipo debe preparar su computadora antes de levantar el proyecto. La mayoría usará Windows 11, por eso los comandos principales están pensados para PowerShell o la terminal integrada de Visual Studio Code.

Herramienta	Para qué sirve
Python	Ejecutar Django y crear el entorno virtual.
pip	Instalar dependencias del proyecto.
Visual Studio Code	Abrir y editar el proyecto.
Git	Clonar el repositorio y subir cambios.
MySQL Community Server	Servidor de base de datos.
MySQL Workbench	Administrar MySQL visualmente, opcional pero recomendado.
Navegador web	Probar el sistema localmente.

Importante: No se debe copiar la carpeta `venv` de una computadora a otra. Cada integrante debe

crear su propio entorno virtual en su equipo.

5. Instalación de herramientas necesarias en Windows 11

5.1 Instalar Python

Instalar Python desde su página oficial. Durante la instalación se debe marcar la casilla Add Python to PATH. Si no se marca, Windows puede no reconocer el comando python.

```
python --version  
pip --version
```

Si ambos comandos muestran una versión, Python y pip quedaron disponibles en el sistema.

5.2 Instalar Visual Studio Code

Instalar Visual Studio Code y abrir el proyecto desde File > Open Folder. También se puede usar la terminal integrada desde Terminal > New Terminal.

5.3 Instalar Git

Git se necesita para clonar el repositorio privado, descargar cambios y subir avances. Después de instalarlo, verificar con:

```
git --version
```

5.4 Instalar MySQL Community Server y Workbench

Instalar MySQL Community Server. Durante la instalación se debe definir una contraseña para el usuario root. Esa contraseña se usará luego en config/settings.py.

Nota: MySQL Workbench no reemplaza al servidor. Workbench solo es una herramienta visual. El servidor MySQL es lo que Django necesita para conectarse a la base de datos.

```
mysql --version
```

6. Clonar el repositorio privado desde GitHub

Antes de clonar, cada integrante debe tener cuenta de GitHub y haber aceptado la invitación al repositorio privado. Si no aceptó la invitación, al intentar clonar puede aparecer Repository not found o Authentication failed.

6.1 Clonar desde terminal

```
git clone https://github.com/anthony-devtecnio/atlantic_city_crm.git  
cd atlantic_city_crm
```

6.2 Abrir el proyecto en VS Code

```
code .
```

Nota: Si el comando code no funciona, abrir Visual Studio Code manualmente y seleccionar la carpeta atlantic_city_crm.

6.3 Revisar la estructura del proyecto

```
dir
```

Se debe ver una estructura parecida a esta:

```
manage.py
config
clientes
visitas
incidencias
promociones
dashboard
templates
static
requirements.txt
.gitignore
```

7. Crear y activar el entorno virtual

El entorno virtual es una carpeta local donde se instalan las dependencias del proyecto. Esto evita mezclar librerías del sistema con las librerías del proyecto.

7.1 Crear entorno virtual

```
python -m venv venv
```

7.2 Activar entorno virtual en Windows

```
venv\Scripts\activate
```

Si se activó correctamente, la terminal debe mostrar algo como:

```
(venv)
```

Importante: Desde este punto, los comandos de pip, Django, check, migrate y runserver deben ejecutarse con el entorno virtual activo.

7.3 Si PowerShell bloquea la activación

En algunos equipos Windows puede aparecer un error de permisos al activar el entorno. Si ocurre, abrir PowerShell como usuario normal y ejecutar:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Luego cerrar y abrir nuevamente la terminal de VS Code y activar otra vez:

```
venv\Scripts\activate
```

8. Instalar dependencias del proyecto

Con el entorno virtual activo, se instalan las librerías necesarias.

```
python -m pip install --upgrade pip setuptools wheel  
pip install -r requirements.txt
```

8.1 Verificar Django

```
django-admin --version
```

8.2 Verificar conector MySQL

```
python -c "import MySQLdb; print('Conector MySQL funcionando')"
```

8.3 Si no existe requirements.txt

Si por algún motivo el archivo requirements.txt no aparece, instalar manualmente:

```
pip install django==6.0.6 mysqlclient
```

8.4 Crear o actualizar requirements.txt

Cuando se instale una nueva dependencia importante, se debe actualizar el archivo:

```
pip freeze > requirements.txt
```

9. Configurar MySQL y crear la base de datos

El proyecto usa MySQL. Cada integrante debe tener su propia base de datos local llamada atlantic_city_crm.

9.1 Entrar a MySQL desde terminal

```
mysql -u root -p
```


Ingresa la contraseña configurada durante la instalación de MySQL.

9.2 Crear base de datos

```
CREATE DATABASE atlantic_city_crm  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```

9.3 Verificar base de datos

```
SHOW DATABASES;  
exit;
```

10. Configurar conexión en settings.py

Abrir el archivo config/settings.py y revisar la sección DATABASES. Cada integrante debe colocar su propia contraseña local de MySQL.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'atlantic_city_crm',  
        'USER': 'root',  
        'PASSWORD': 'TU_CONTRASEÑA_LOCAL',  
        'HOST': '127.0.0.1',  
        'PORT': '3306',  
        'OPTIONS': {  
            'charset': 'utf8mb4',  
        },  
    }  
}
```

Importante: La contraseña de MySQL no tiene que ser igual para todos. Lo importante es que cada uno coloque en settings.py la contraseña que configuró en su propio equipo.

11. Verificar, migrar y levantar el sistema

Antes de crear tablas o ejecutar el servidor, siempre se debe verificar el proyecto.

11.1 Verificar el proyecto

```
python manage.py check
```

Resultado esperado:

```
System check identified no issues (0 silenced).
```

Importante: Si check muestra errores, no ejecutar migrate ni runserver. Primero se debe resolver el error.

11.2 Aplicar migraciones

```
python manage.py migrate
```

Este comando crea las tablas necesarias en MySQL.

11.3 Verificar tablas en MySQL

```
mysql -u root -p
USE atlantic_city_crm;
SHOW TABLES;
exit;
```

Deben aparecer tablas como:

```
auth_user
auth_group
django_session
clientes_cliente
visitas_visita
incidencias_incidencia
promociones_promocion
promociones_clientepromocion
```

11.4 Levantar servidor

```
python manage.py runserver
```

Rutas principales:

```
Sistema: http://127.0.0.1:8000/login/
Dashboard: http://127.0.0.1:8000/dashboard/
Admin: http://127.0.0.1:8000/admin/
```

12. Crear superusuario e ingresar al administrador de Django

Como la base de datos se crea localmente, cada integrante debe crear su propio superusuario.

```
python manage.py createsuperuser
```

Datos sugeridos para entorno de prueba:

Username: admin
 Email: admin@atlanticcity.com
 Password: colocar una contraseña segura

***Nota:** La contraseña no se muestra mientras se escribe. Eso es normal.*

12.1 Entrar al administrador

<http://127.0.0.1:8000/admin/>

Desde Django Admin podemos crear usuarios, grupos, permisos, promociones y datos de prueba.

13. Crear grupos, roles y permisos en Django Admin

Dentro del administrador de Django se deben crear los grupos que usamos para controlar permisos dentro del sistema.

13.1 Grupos necesarios

- Administrador
- Operaciones
- Marketing
- AtencionCliente

13.2 Permisos sugeridos por grupo

Grupo	Permisos recomendados
Administrador	Permisos completos sobre clientes, visitas, incidencias, promociones, usuarios y grupos.
Operaciones	Ver, agregar y editar clientes. Ver, agregar y editar visitas. Ver, agregar y editar incidencias.
Marketing	Ver clientes. Ver, agregar y editar promociones. Ver, agregar y editar promociones asignadas.
AtencionCliente	Ver clientes. Ver, agregar y editar incidencias. Ver y agregar visitas.

13.3 Asignar grupo a un usuario

1. Entrar a Users en Django Admin.
2. Abrir el usuario correspondiente.
3. Buscar la sección Groups.
4. Seleccionar el grupo que corresponde.
5. Guardar cambios.
6. Verificar que el usuario esté activo.

Importante: Si un usuario no tiene permiso para una acción, puede no ver el botón correspondiente o puede recibir una página 403. Esto significa que la seguridad del sistema está funcionando.

14. Crear datos de prueba y promociones

Para probar el sistema se recomienda crear datos mínimos: un cliente, una visita, una incidencia y varias promociones. Luego se debe asignar al menos una promoción a un cliente.

14.1 Crear cliente

Se puede crear desde el sistema en /clientes/ o desde Django Admin. Datos sugeridos:

DNI: 12345678
 Nombres: Carlos
 Apellidos: Ramirez Torres
 Telefono: 999888777
 Correo: carlos.ramirez@email.com
 Direccion: Lima
 Estado: Activo

14.2 Crear visita

Cliente: seleccionar cliente existente
 Motivo: Juego
 Observacion: Cliente ingreso por promocion de bienvenida.

14.3 Crear incidencia

Cliente: seleccionar cliente existente
 Tipo: Reclamo
 Asunto: Demora en atencion
 Descripcion: El cliente reporto demora en el proceso de atencion.
 Estado: Pendiente
 Respuesta: dejar vacio si aun no se atiende.

14.4 Crear promociones desde Django Admin

Entrar a Promociones en Django Admin y crear estas promociones de prueba:

Nombre	Descripción	Estado
Bono de bienvenida	Promoción dirigida a clientes nuevos registrados en el sistema.	Activa
Promoción cliente frecuente	Beneficio para clientes que registran visitas recurrentes.	Activa
Campaña especial de fin de semana	Promoción temporal para incentivar participación durante fines de semana.	Activa

14.5 Asignar promoción a un cliente

Desde Django Admin o desde la ficha del cliente, asignar una promoción:

Cliente: seleccionar cliente existente
Promocion: Bono de bienvenida
Estado: Asignada
Observacion: Promocion asignada por registro inicial del cliente.

15. Uso básico del sistema

15.1 Iniciar sesión

<http://127.0.0.1:8000/login/>

Ingresar usuario y contraseña. Después del login, el sistema redirige al dashboard.

15.2 Dashboard

El dashboard muestra totales de clientes, visitas, incidencias y promociones. También muestra estados de clientes, incidencias y promociones, además de últimos clientes y últimas visitas.

15.3 Clientes

En el módulo Clientes se puede listar, buscar, registrar y editar clientes. La ficha única permite consultar todo el historial relacionado con un cliente.

<http://127.0.0.1:8000/clientes/>

15.4 Ficha única del cliente

Desde la ficha del cliente se pueden registrar visitas, registrar incidencias y asignar promociones. También se visualizan los contadores y los últimos registros asociados.

15.5 Visitas

<http://127.0.0.1:8000/visitas/>

Permite revisar todas las visitas registradas y buscar por cliente, DNI, correo, motivo u observación.

15.6 Incidencias

<http://127.0.0.1:8000/incidencias/>

Permite revisar incidencias, buscar registros y ver el cliente asociado. Los estados principales son pendiente, en proceso, resuelta y cerrada.

15.7 Promociones

```
http://127.0.0.1:8000/promociones/
```

Permite ver promociones disponibles. La creación de promociones se puede realizar desde Django Admin y la asignación se puede hacer desde la ficha del cliente.

16. Trabajo con Git y GitHub

GitHub privado será la forma oficial de compartir el código. Para evitar errores, el equipo debe seguir una forma de trabajo simple y ordenada.

16.1 Revisar estado del proyecto

```
git status
```

16.2 Descargar cambios del repositorio

```
git pull origin main
```

Importante: Antes de empezar a trabajar, conviene ejecutar `git pull origin main` para tener la última versión.

16.3 Agregar cambios

```
git add .
```

16.4 Crear commit

```
git commit -m "Descripcion clara del cambio"
```

Ejemplos de mensajes de commit:

```
git commit -m "Actualizar README del proyecto"
git commit -m "Corregir formulario de clientes"
git commit -m "Agregar capturas para documentacion"
```

16.5 Subir cambios a GitHub

```
git push origin main
```

16.6 Flujo recomendado antes de subir cambios

7. Activar el entorno virtual.
8. Ejecutar `python manage.py check`.

9. Probar el módulo que se modificó.
10. Ejecutar git status.
11. Agregar archivos con git add .
12. Crear commit con mensaje claro.
13. Subir con git push origin main.

```

venv\Scripts\activate
python manage.py check
git status
git add .
git commit -m "Mensaje claro"
git push origin main

```

16.7 Qué hacer si aparece Repository not found

Este error suele aparecer cuando el integrante no aceptó la invitación, no inició sesión en GitHub o no tiene permiso para el repositorio privado.

- Confirmar que el dueño del repositorio envió la invitación.
- Aceptar la invitación desde GitHub o desde el correo.
- Verificar que se inició sesión con la cuenta correcta.
- Intentar clonar nuevamente.

16.8 Reglas internas para no romper el proyecto

- No borrar migraciones sin avisar.
- No subir venv.
- No cambiar el nombre de la base de datos.
- No modificar settings.py con contraseñas de otra persona.
- No hacer push si el proyecto no pasa python manage.py check.
- Avisar al equipo antes de modificar archivos centrales como settings.py o urls.py.

17. Errores comunes y soluciones

Error	Posible causa	Solución
No module named django	El entorno virtual no está activo o no se instaló Django.	Activar venv e instalar dependencias.
Unknown database atlantic_city_crm	No se creó la base de datos en MySQL.	Crear la base de datos con CREATE DATABASE.
Access denied for user root	Contraseña de MySQL incorrecta.	Revisar PASSWORD en settings.py.
mysqlclient no instala	Faltan dependencias o compiladores en Windows.	Revisar instalación de MySQL y dependencias; instalar herramientas necesarias.
Repository not found	No hay acceso al repositorio privado.	Aceptar invitación o iniciar sesión con la cuenta correcta.
403 Forbidden	El usuario no tiene permiso para esa acción.	Asignar permisos o usar un usuario con el rol correcto.
El menú no muestra una opción	El usuario no tiene permiso para ver ese módulo.	Revisar grupos y permisos en Django Admin.

runserver no levanta	Hay error previo en configuración o dependencias.	Ejecutar <code>python manage.py check</code> y corregir primero.
----------------------	---	--

```
# Comandos rápidos para revisar el proyecto
venv\Scripts\activate
python manage.py check
python manage.py migrate
python manage.py runserver
```

18. Nota para quien trabaja en Mac

El equipo trabajará principalmente en Windows 11, pero si se usa Mac, algunos comandos cambian. Esto lo dejo para evitar confusiones.

Acción	Windows	Mac
Activar entorno	<code>venv\Scripts\activate</code>	<code>source venv/bin/activate</code>
Listar archivos	<code>dir</code>	<code>ls</code>
Limpiar carpeta venv	<code>rmdir /s /q venv</code>	<code>rm -rf venv</code>
Servidor Django	<code>python manage.py runserver</code>	<code>python manage.py runserver</code>

Nota: En Mac, si `mysqlclient` no encuentra `libmysqlclient`, puede ser necesario configurar `DYLD_LIBRARY_PATH`. Esto no aplica necesariamente para Windows.

19. Checklist final para levantar el proyecto

- ☐ Tengo acceso al repositorio privado en GitHub.
- ☐ Cloné el repositorio en mi equipo.
- ☐ Abrí el proyecto en Visual Studio Code.
- ☐ Instalé Python y pip funcionan.
- ☐ Instalé MySQL y tengo contraseña root.
- ☐ Creé la base de datos `atlantic_city_crm`.
- ☐ Creé y activé el entorno virtual.
- ☐ Instalé dependencias desde `requirements.txt`.
- ☐ Configuré la contraseña de MySQL en `settings.py`.
- ☐ Ejecuté `python manage.py check` sin errores.
- ☐ Ejecuté `python manage.py migrate`.
- ☐ Creé superusuario.
- ☐ Entré al admin de Django.
- ☐ Creé grupos y permisos.
- ☐ Levanté el servidor con `runserver`.
- ☐ Entré al sistema desde `/login/`.
- ☐ Probé dashboard, clientes, visitas, incidencias y promociones.

20. Cierre y recomendaciones finales

Este manual debe servir como guía central para el equipo. La intención es que cada integrante pueda levantar el sistema y trabajar sin depender de una sola computadora. Si algo falla, la primera recomendación es revisar el mensaje de error, ejecutar `python manage.py check` y confirmar que el entorno virtual esté activo.

También recomiendo que antes de cualquier avance importante se haga `git pull origin main`, y antes de subir cambios se pruebe el sistema localmente. Esto nos ayudará a evitar conflictos y a mantener el proyecto estable para la presentación final.

Importante: No se debe esperar al final para probar. Cada cambio debe validarse de inmediato. Si el proyecto pasa check, migra correctamente y levanta el servidor, vamos por buen camino.

Referencias de apoyo

Estas referencias oficiales se pueden consultar si el equipo necesita revisar el uso de repositorios, colaboradores o clonación en GitHub.

Fuente	Tema
GitHub Docs	Quickstart for repositories: creación y administración de repositorios.
GitHub Docs	Inviting collaborators to a personal repository: invitación de colaboradores.
GitHub Docs	Cloning a repository: clonación de repositorios.
GitHub Docs	Permission levels for a personal account repository: permisos de colaboradores.

En caso de duda con GitHub, consultar la documentación oficial desde docs.github.com. La interfaz puede cambiar con el tiempo, pero las ideas principales se mantienen: crear repositorio, invitar colaboradores, clonar, hacer commits y subir cambios.